Leading a Capstone Team

A handover

Notes for the next student leads

Jack Montgomery-Parkes :: S223232288@deakin.edu.au

Why this exists

By the time you have your bearings in this unit, you're likely already a third of the way through it.

This deck is a short handover from someone who has worked in project management for years. It is not a manual, and it is not the only way to do this. It is a set of things that work for me, with the reasoning kept in.

Take what fits, ignore what doesn't. Each team is different, but there are tools that can help.

Part one

Team Creation

& communication

Start with the brief, not the work

Most task briefs in this unit are written for whoever wrote them. That's fine, but it leaves a translation step.

Before you allocate anything, rewrite the brief in your own words, for the person with the least context on your team. If you can't, you don't understand it yet either.

Assume the lowest common denominator, generously. Plain language is not condescending, unclear language is.

Rewritten Brief Example

Old brief

Action: Analyse the behaviour of the selected production model on representative test and real-world audio samples to identify where it performs well and where it fails. This should include class-wise error patterns, confusion between similar species, confidence behaviour on incorrect predictions, and any recurring weaknesses that could affect the working Echo system.

Deliverable: A detailed error analysis report for the selected production model including common misclassification patterns, species-level or class-level confusion trends, confidence behaviour on correct and incorrect predictions, likely causes of major errors, and practical recommendations for Sprint 3 improvements.

Rewritten brief

What we're doing

Run the production model over a stratified set of Echo audio clips and record where it gets things wrong.

What you're delivering

A short report, with confusion matrix, 3–5 example failure cases, and next step recommendations.

Who to ask

[previous student] for model access, Engine lead for dataset and Github access.

Cadence beats ceremony

Long, infrequent meetings produce minutes nobody reads. Short, regular check-ins produce a team that actually knows what's happening.

For me this usually looks like one of three things:

Check-ins

Short, minute or two per person, weekly. Focus on "what's in your way, what do you need from me". Could be a call, could be an e-mail or Teams message. Just keep it brief.

Group updates

Short written summary after each official meeting. What was decided, who's doing what, what's still open. People miss meetings; they shouldn't miss decisions.

Upwards relay

Pass information from the unit chair / leadership back to the team in plain terms. People work better when they know why something matters.

Part two

Identifying needs

& delegating tasks

Match work to people

Allocation is the part most people skip. They look at the task list, look at the team list, and pair them off in order.

Take an extra five minutes per person. Ask:

- What are they actually good at, beyond what the unit assumes?
- What do they want to get better at this trimester?
- What else are they carrying: other units, work, life?
- Which tasks would stretch them without breaking them?

Each person is going to have a different amount they can give. Pushing against that too much can have knock-on effects that delay progress and waste time.

Dependency charts unlock parallel work

Most sprints look like a list of tasks. They're more usually a graph.

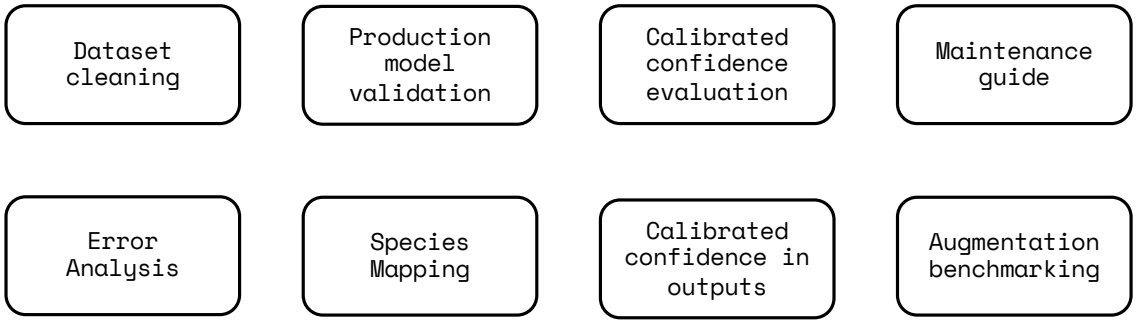
Spend twenty minutes drawing it. Two things drop out:

- What can run in parallel (most things, usually).
- What's actually on the critical path (fewer things than you'd think).

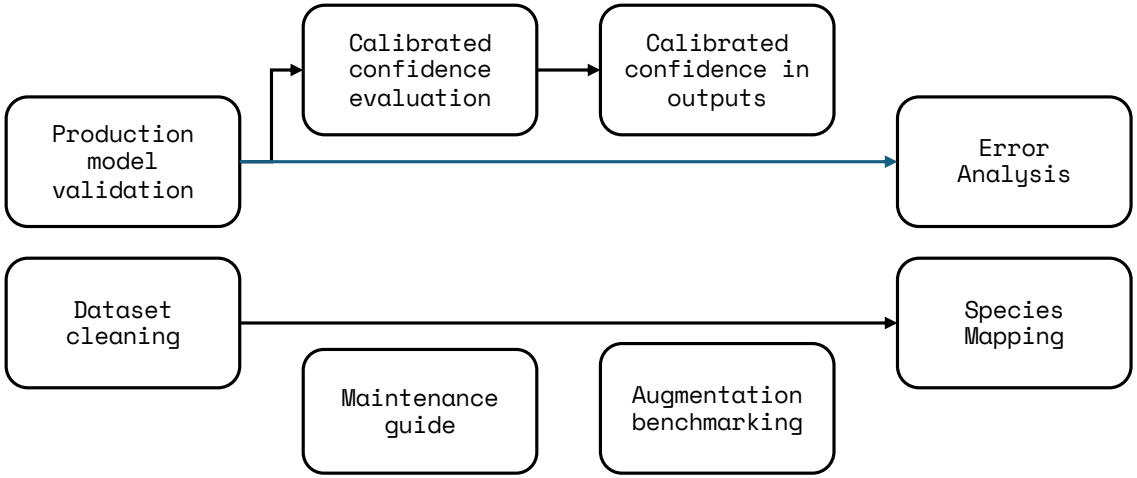
You're going to have some students who are happy to do as little as possible, and you're going to have some power nerds.

Making sure you know what the critical path is means you can allocate tasks without blocking, while making sure everyone gets to build their skills.

Sprint Allocation



Dependency Graph



**Can be completed at any time*

Part three

Progress tracking

individual & group

Status should be discoverable

The single most important thing you can do for a team is make it so nobody has to ask where anything is.

If your progress tracker needs explaining, it isn't tracking progress. It's just getting in the way. We have access to Microsoft Loop through our university accounts, Kanban boards are your friend so long as they're actively maintained.

Dataset cleaning
Owner: Person A
Starts immediately; unblocks ALA Species Mapping.
Due: *5 Jun 2026*

Production model validation
Owner: Person B
Starts immediately; unblocks Calibrated Confidence Eval, Engine-IoT Validation, TFLite in Production Path, and Error Analysis.
Due: *12 Jun 2026*

Calibrated confidence evaluation
Owner: Person C
Depends on Production Model Validation; unblocks Calibrated Confidence in Outputs.
Due: *26 Jun 2026*

Maintenance guide
Owner: Person D
Starts immediately; no downstream dependencies.
Due: *10 Jul 2026*

What it comes down to

- Clarity is a kindness.
- Small, regular contact beats big, occasional contact.
- Match work to people.
- Make status discoverable, not askable.
- The team is students. You're going to be dealing with so many kinds of people. Lead accordingly.